
IA

Clasificador de regimen

G1

Andrés Felipe Higuera Graterón

Objetivo

Se busca clasificar el regimen el régimen cambiario de una divisa,este una propiedad del país emisor de la moneda e incluye categorías como:

- Free Float (flotación libre) - 66.13%
- Managed Float (flotación administrada) - 23.26%
- High Intervention (alta intervención institucional)-8.67%
- Fixed Peg (tipo de cambio fijo) - 1.93 %

Data base

Se construye un dataset de características estadísticas a partir de datos de alta frecuencia del mercado de divisas por medio de API de MT5.

Se construyen las monedas sinteticas segun su fuerza relativa sobre las cuales se trabajaron

Las monedas construidas fueron:

- **USD (Dólar estadounidense)**
- **EUR (Euro)**
- **JPY (Yen japonés)**
- **GBP (Libra esterlina)**
- **CHF (Franco suizo)**
- **CAD (Dólar canadiense)**
- **AUD (Dólar australiano)**
- **NZD (Dólar neozelandés)**

Estructura y dimension

moneda	mean_return	std_return	skewness	kurtosis	trend_slope	mean_volume	extreme_count
EUR	...	...	...	...	...	...	...
EUR	...	...	...	...	...	...	...
USD	...	...	...	...	...	...	...

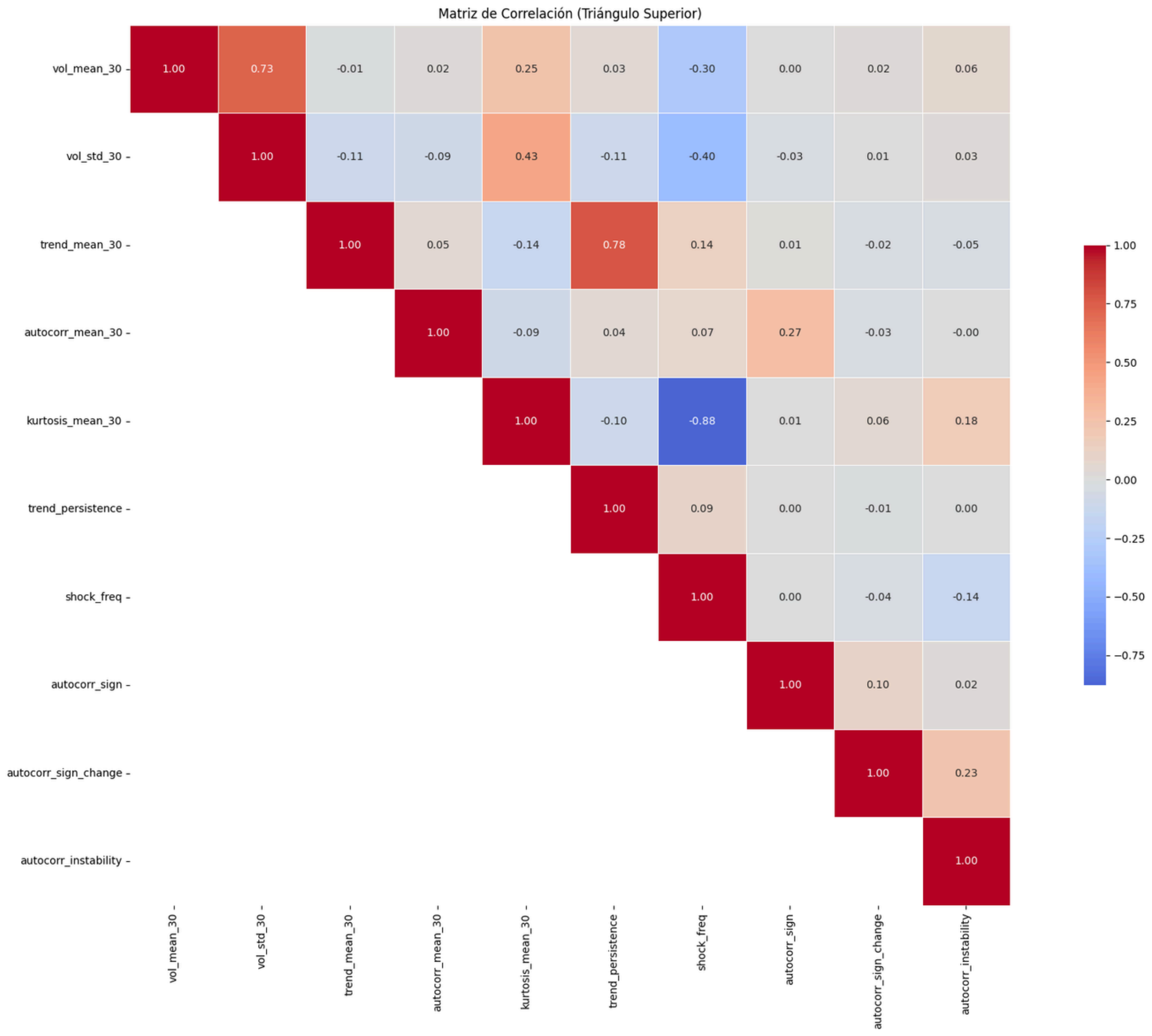
```
Shape X: (8533, 11)
Train: (6826, 11) Test: (1707, 11)
```

Variables

Los datos secuenciales se analizaron y agruparon para eliminar dependencia temporal y construir variables que permitieran describir comportamientos no lineales comunes de la naturaleza de los datos. Como:

- Tendencia
- Volatilidad
- Persistencia temporal
- Actividad de mercado
- Estructura del movimiento

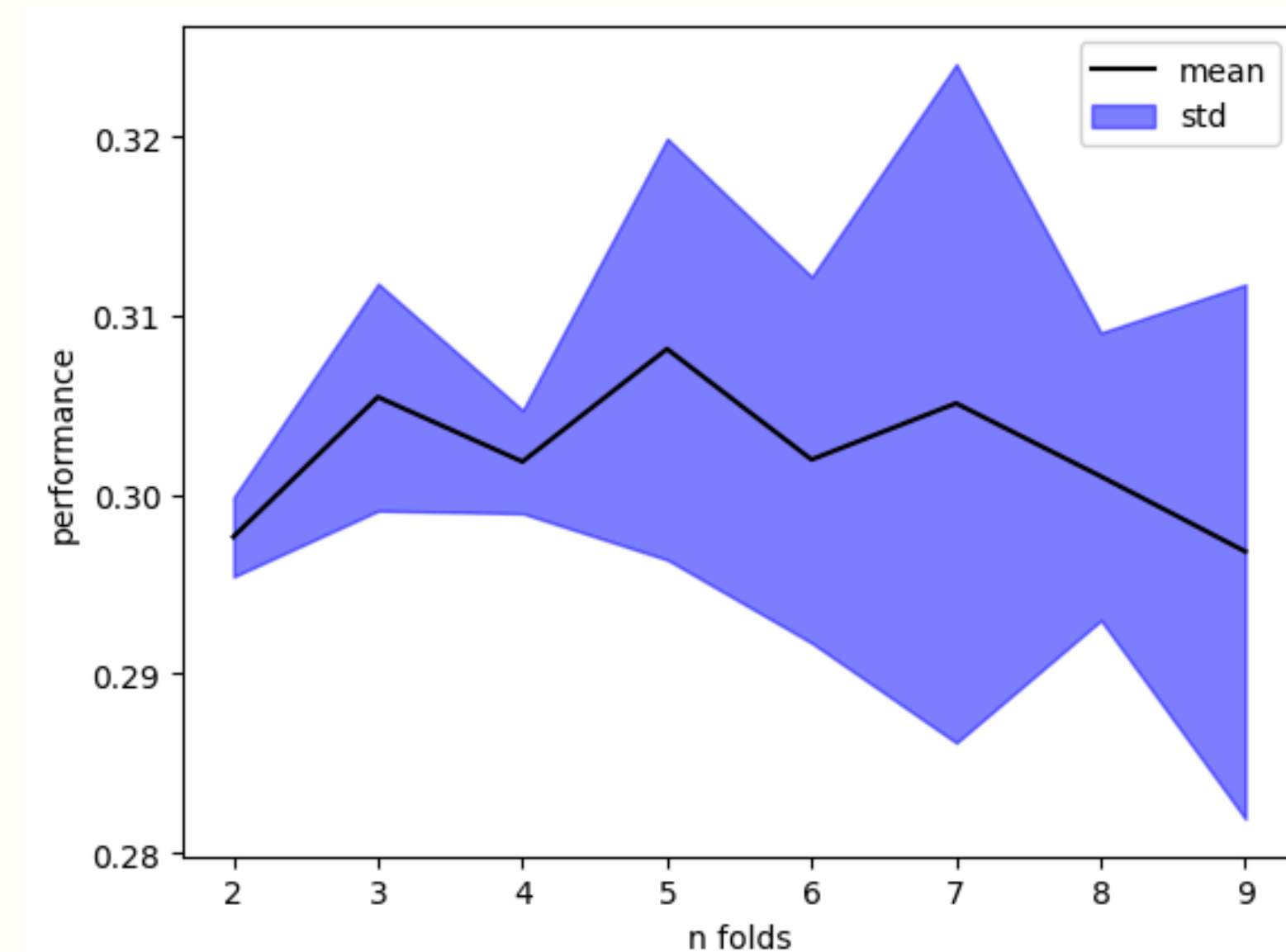
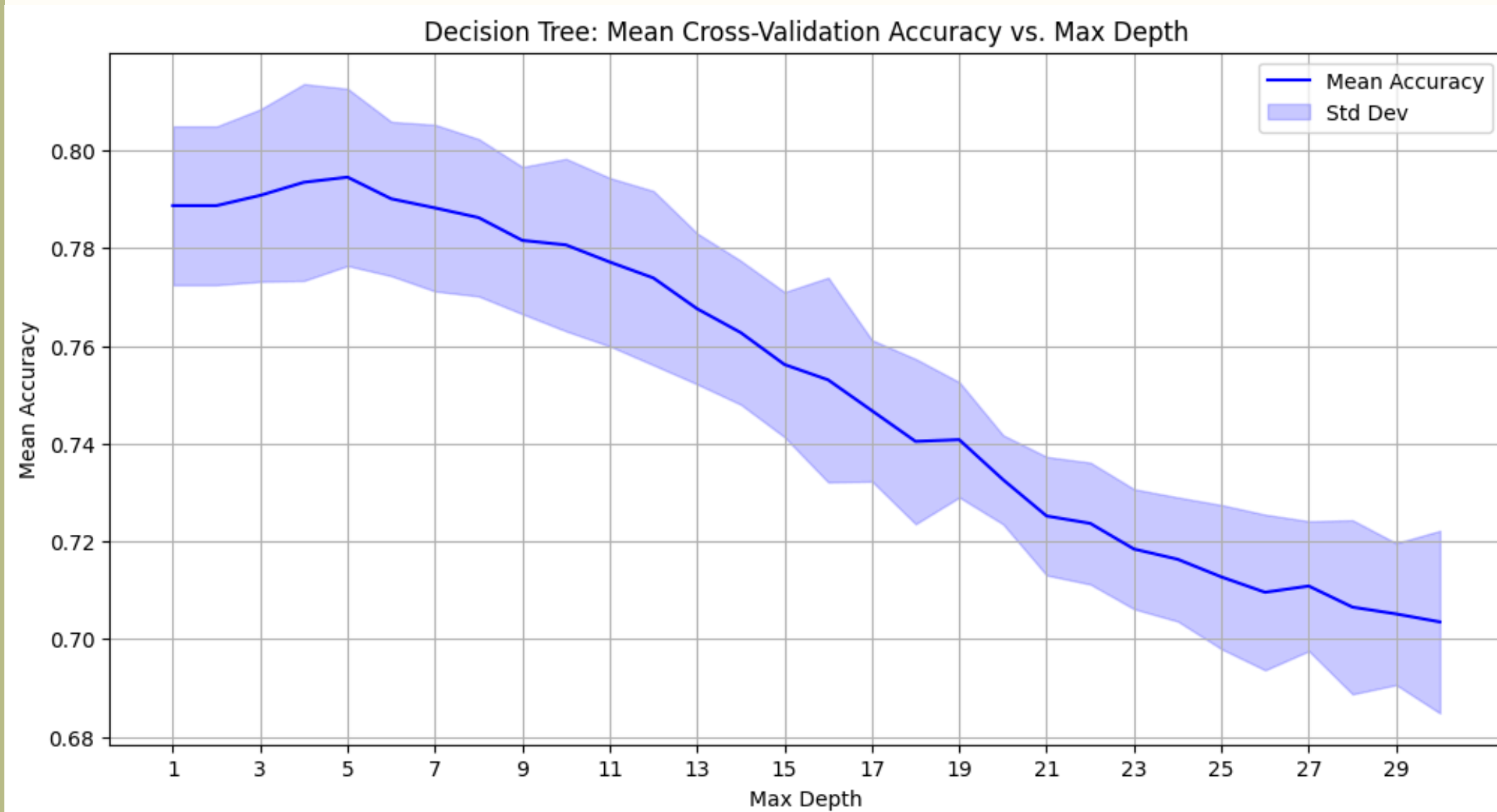
Variable	Descripción
kurtosis	Curtosis de la distribución de retornos
vol_mean_30	Media de la volatilidad en ventana de 30 días
vol_std_30	Desviación estándar de la volatilidad en 30 días
trend_mean_30	Media de la tendencia en ventana de 30 días
autocorr_mean_30	Media de la autocorrelación en 30 días
autocorr_std_30	Desviación estándar de la autocorrelación en 30 días
trend_persistence_30	Persistencia de la tendencia en 30 días
autocorr_decay	Tasa de decaimiento de la autocorrelación
max_drawdown	Máxima reducción de precios
autocorr_sign	Signo de la autocorrelación
autocorr_sign_change	Cambios de signo en la autocorrelación
autocorr_instability	Inestabilidad de la autocorrelación



Modelo: Decision Tree DT

DT Se implemento el modelo DT , del cual obtuvo el rendimiento mas eficiente con profundidad (max_depth) de 5 y con un cross_validation optimo de 5 n folds.

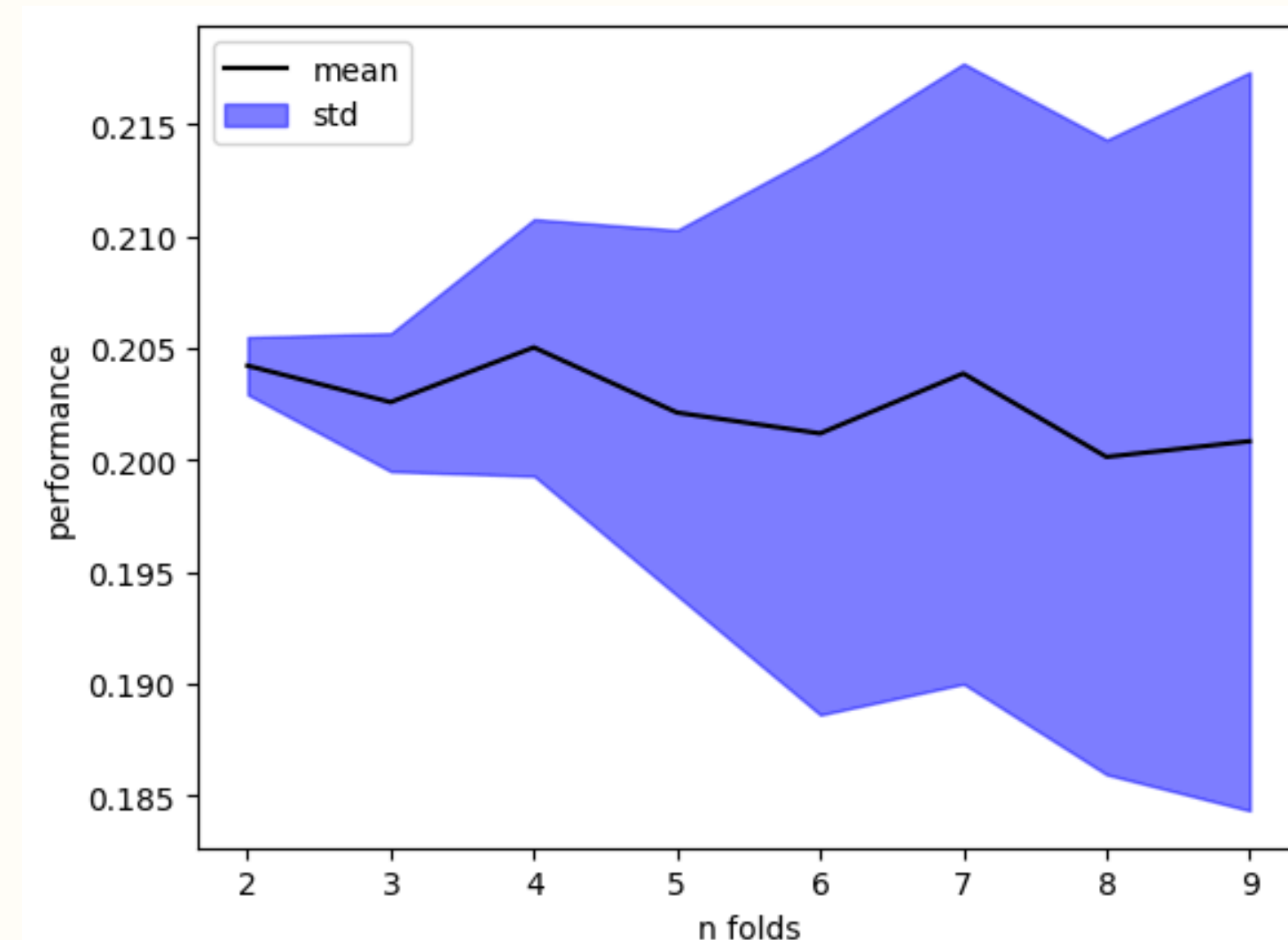
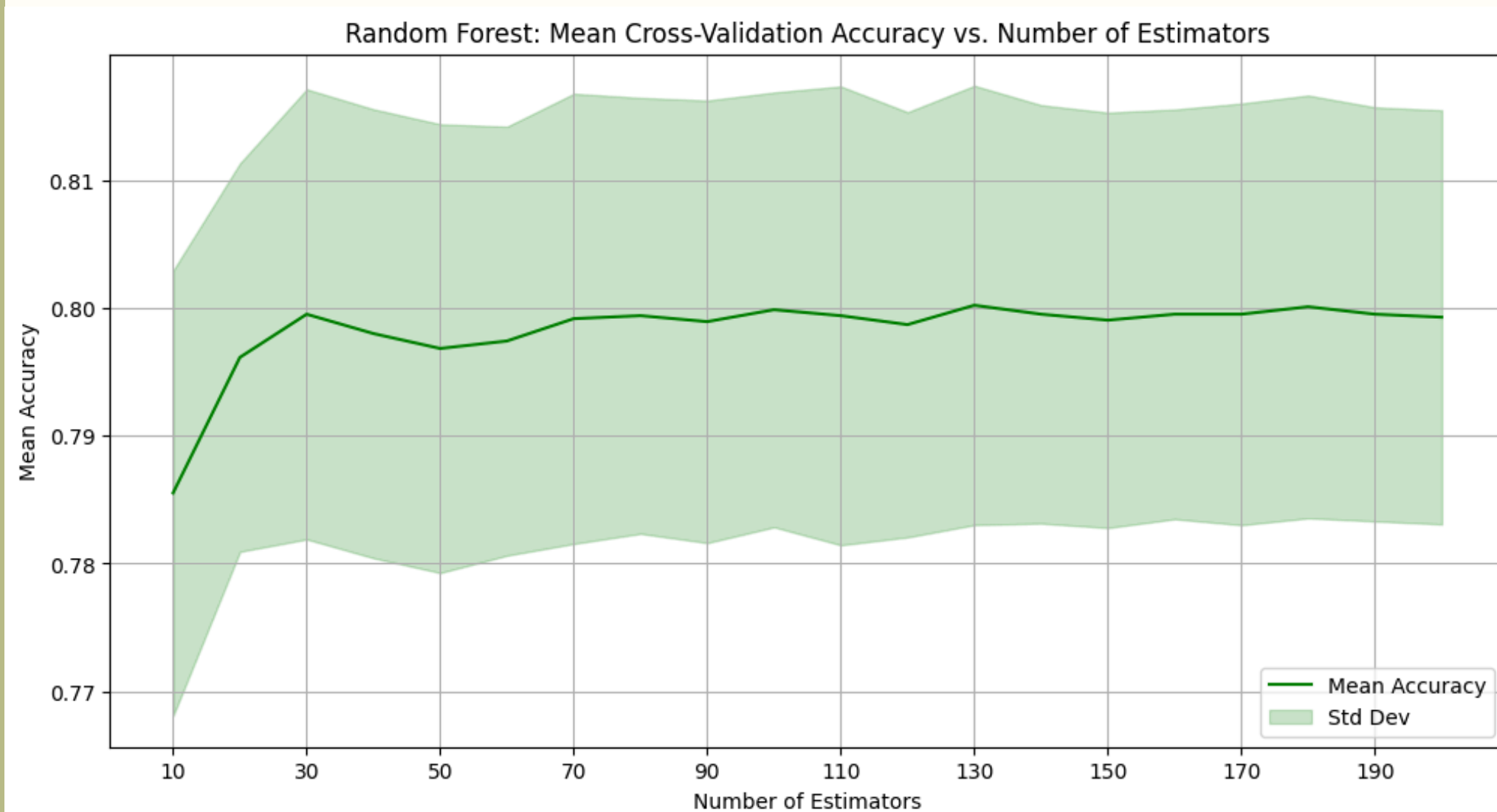
Se obtuvo un accuracy del 0.791% (+/- 0.00866)



Modelo: RandomForest RF

RF Se implemento el modelo RF , del cual se obtuvo un rendimiento mas eficiente con 130 estimadores con un cross_validation optimo de 4 nfolds.

Se obtuvo un accuracy del 0.801% (+/- 0.00811)



Modelo: Support Vectorial Machine SVM

Se implemento SVM con sus 3 diferentes kernels con modificaciones para acelerar convergencia.

Los 3 Kernels obtuvieron el mismo rendimiento: 0.769% (+/- 0.00893)

Kernel Lineal

Kernel RBF

Kernel poly

```
SVM linear

#@title SVM linear
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, KFold
import numpy as np

# Configuración de validación común
kf = KFold(n_splits=5, shuffle=True, random_state=42)
est_linear = make_pipeline(StandardScaler(), SVC(kernel="linear", cache_size=1000))
s_linear = cross_val_score(est_linear, X, y, cv=kf, n_jobs=-1)

print(f" Resultado Lineal: {np.mean(s_linear):.3f} (+/- {np.std(s_linear):.5f})"

... Resultado Lineal: 0.769 (+/- 0.00893)
```

```
SVC rbf

#@title SVC rbf
#@title SVM linear
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, KFold
import numpy as np

# Configuración de validación común
kf = KFold(n_splits=5, shuffle=True, random_state=42)
est_linear = make_pipeline(StandardScaler(), SVC(kernel="rbf", cache_size=1000))
s_linear = cross_val_score(est_linear, X, y, cv=kf, n_jobs=-1)

print(f" Resultado Lineal: {np.mean(s_linear):.3f} (+/- {np.std(s_linear):.5f})"

Resultado Lineal: 0.769 (+/- 0.00893)
```

```
SVC poly

#@title SVC poly
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, KFold
import numpy as np

# Configuración de validación común
kf = KFold(n_splits=5, shuffle=True, random_state=42)
est_linear = make_pipeline(StandardScaler(), SVC(kernel="poly", cache_size=1000))
s_linear = cross_val_score(est_linear, X, y, cv=kf, n_jobs=-1)

print(f" Resultado Lineal: {np.mean(s_linear):.3f} (+/- {np.std(s_linear):.5f})"

Resultado Lineal: 0.769 (+/- 0.00893)
```


Modelo: Deep learning DNN

Se implementaron 2 modelos DNN

- 1er modelo: simple de 4 capas de 128,128,64,4
- 2do modelo: mas complejo de 7 capas con 64,64,128,128,256,256,4

Los cuales obtuvieron un rendimiento de 64% a 84% respectivamente

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_8"
```

Layer (type)	Output Shape	Param #
dense_32 (Dense)	(None, 128)	1,408
dense_33 (Dense)	(None, 128)	16,512
dense_34 (Dense)	(None, 64)	8,256
dense_35 (Dense)	(None, 4)	260

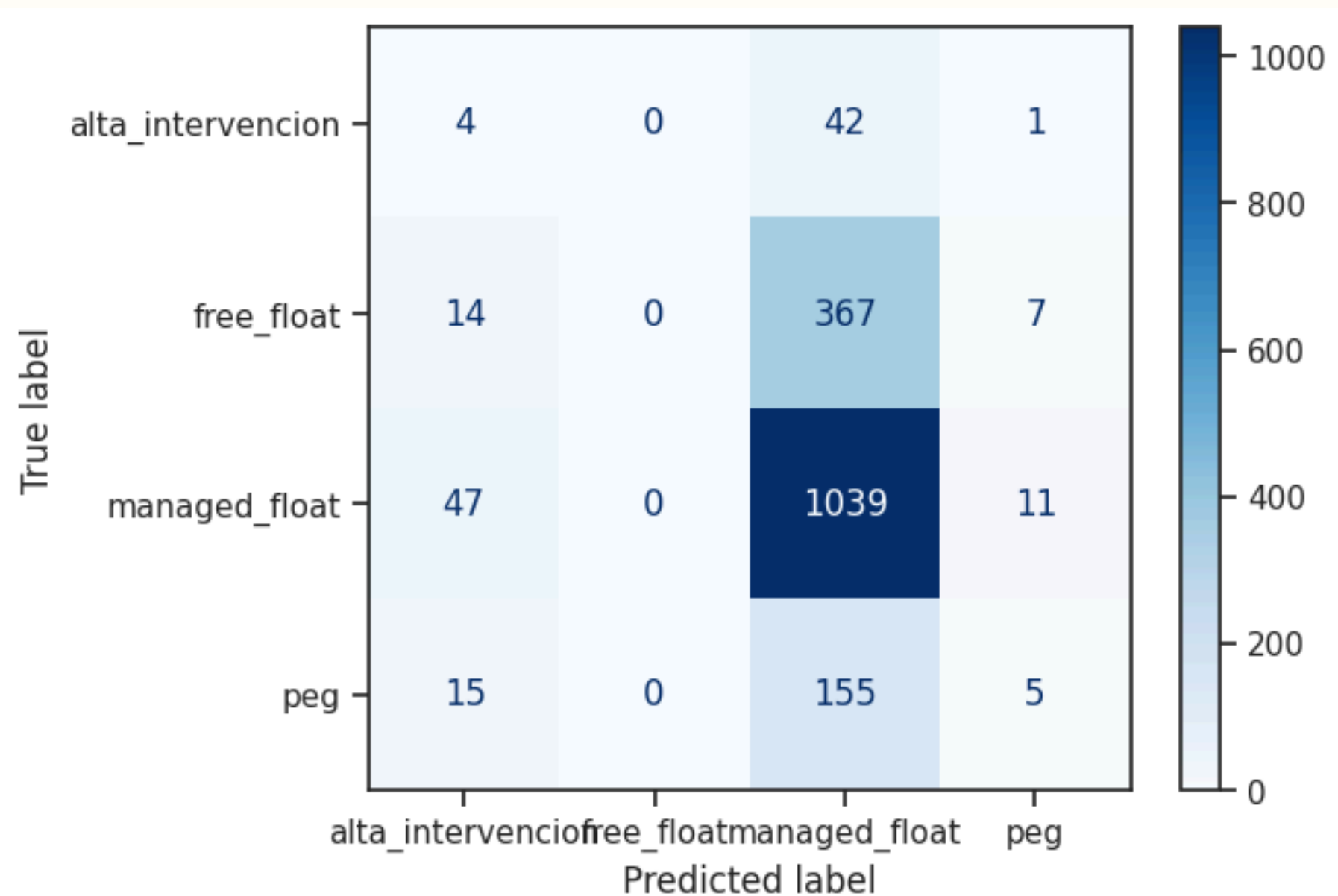
Total params: 26,436 (103.27 KB)
Trainable params: 26,436 (103.27 KB)
Non-trainable params: 0 (0.00 B)

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	768
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 128)	8,320
dense_3 (Dense)	(None, 128)	16,512
dense_4 (Dense)	(None, 256)	33,024
dense_5 (Dense)	(None, 256)	65,792
dense_6 (Dense)	(None, 4)	1,028

Total params: 129,604 (506.27 KB)
Trainable params: 129,604 (506.27 KB)
Non-trainable params: 0 (0.00 B)

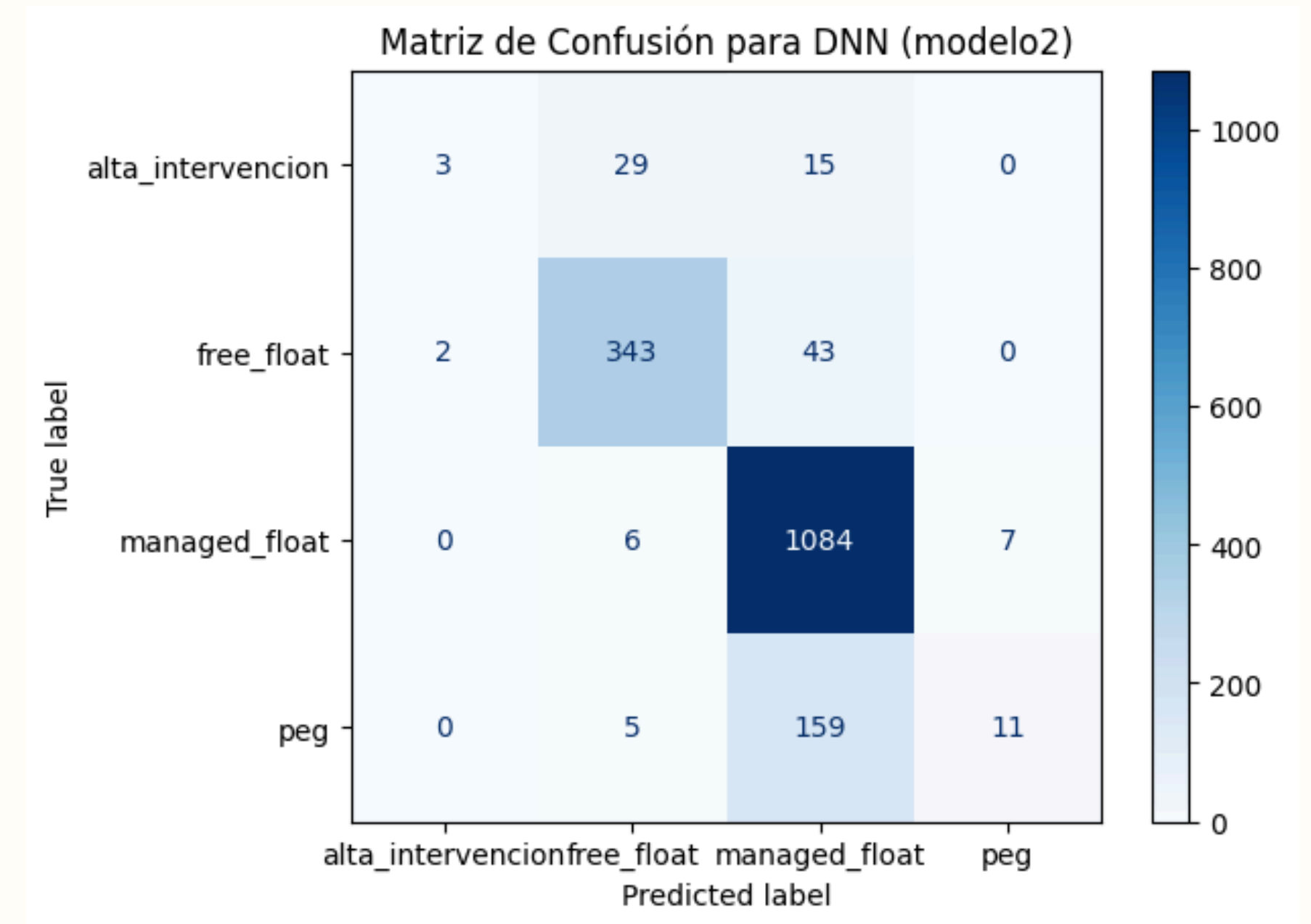
Modelo 1

C0 (alta_intervencion) - 8.51% efect
C1 (free_float) - 0.00% efect
C2 (managed_float) - 94.71% efect
C3 (peg) -2.86% efect

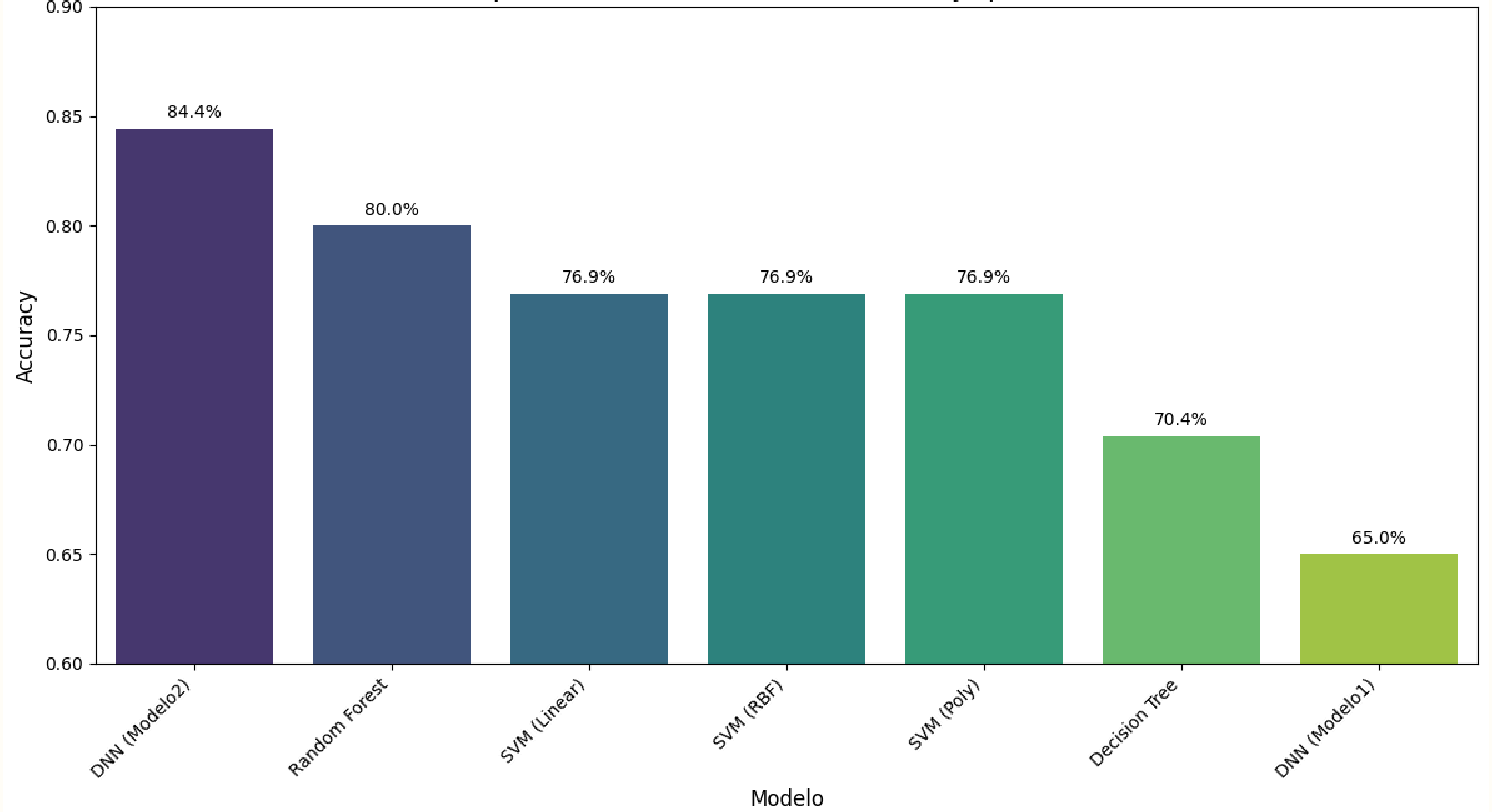


Modelo 2

C0 (alta_intervencion) - 6.38% efect
C1 (free_float) - 88.40% efect
C2 (managed_float) - 98.81% efect
C3 (peg) - 6.29% efect



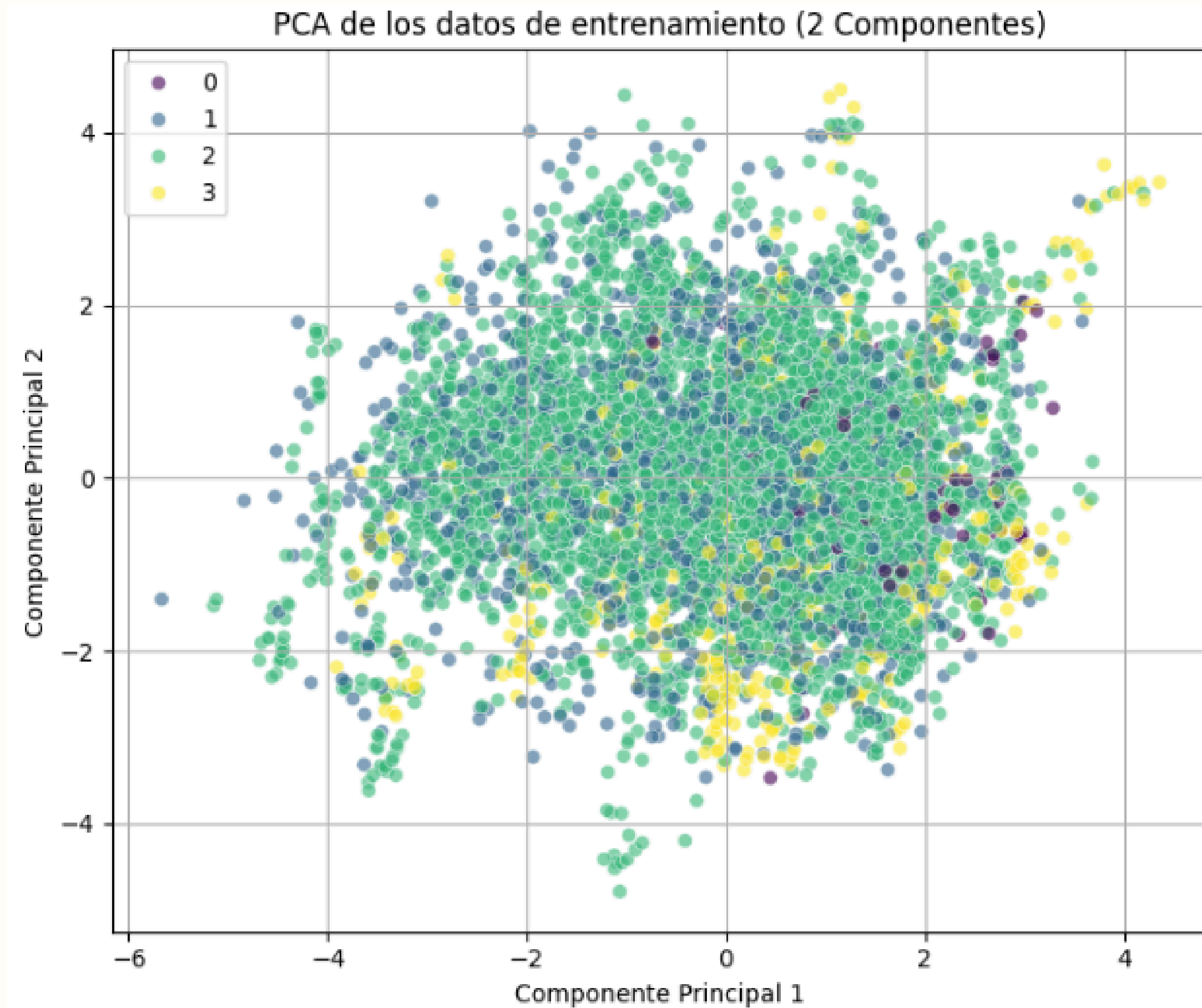
Comparación de Efectividad (Accuracy) por Modelo



Modelo: Machine Learning

Se aplico el metodo PLC para aplanar dimensiones sin perder información en el db.

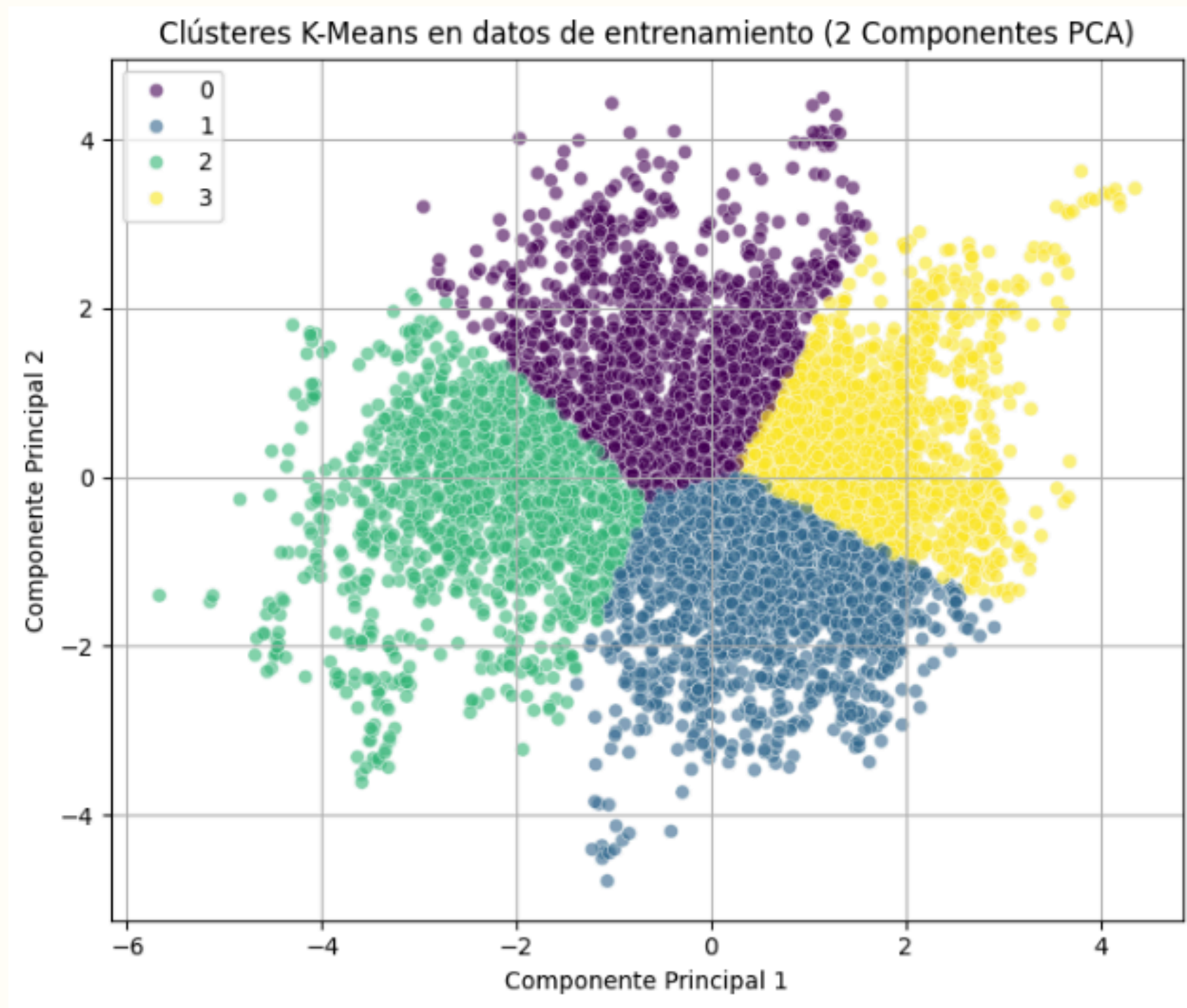
Se puede apreciar como las clases quedan superpuestas sugiriendo un pesimo desarrollo en los metodos K-means y DBSCAN



Modelo: Machine Learning

Se aplico el metodo PLC para aplanar dimensiones pero tenemos que los metodos no son adecuados para las propiedades del db

K-Means



DBSCAN

